

Variable length arrays

A typical problem in programming is the lack of knowledge of limits—particularly the paths and filenames. GCC comes up with a simple and elegant solution for it. Consider the following code snippet:

```
FILE* OpenFile(const char *path, const char* filename, const char*
mode)
{
    char buffer[strlen(path) + strlen(filename) + 1];
    strcpy(buffer, path);
    strcat(buffer, filename);
    return fopen(buffer, mode);
}
```

The above works seamlessly on GCC and produces the expected results.



Note: A more portable way of doing this is to use the `alloca()` function. The syntax is similar to `malloc`, but allocates the memory on a stack frame of the caller. The memory is released when the caller function returns.

Optimisation on likelihood

Consider the following function:

```
extern unsigned int g_event; /* Set to true if event has occurred */
unsigned int my_poll()
{
    static int OccurCount = 0;
    static int FailCount = 0;
    if(g_event)
    {
        /* Handle the event */
        OccurCount += 1;
    }
    else
    {
        /* Handle if the event has not occurred */
        FailCount += 1;
    }
}
```

Now the code for `if(g_event)` in the assembly with `-O2` optimisation looks like the following:

```
a: 84 c0        test    %al,%al
c: 74 12        je     20<, _my_poll+0x20>
```

So the compiler assumes that the value of `g_event` is likely to be non-zero and generates a branch instruction when `g_event` is zero.

This adds a branch penalty if the `g_event` is unlikely and the `else` part is the usual path. This could be communicated to the compiler as follows:

```
#define likely(x)    __builtin_expect((x),1)
#define unlikely(x)  __builtin_expect((x),0)

extern unsigned int g_event; /* Set to true if event has occurred */
unsigned int my_poll()
{
    static int OccurCount = 0;
    static int FailCount = 0;
    if(unlikely(g_event))
    {
        /* Handle the event */
    }
```

And we tell the compiler that `g_event` is a rare event. So the assembly changes to:

```
a: 84 c0        test    %al,%al
c: 75 0b        jne    19<, _my_poll+0x19>
```

This achieves a great improvement in performance.

It will be good to note once again that all these little functionalities offered by GCC are not standard. The code written in such a way is not guaranteed to work on another C compiler. But as long as we stick to GCC, all this will definitely work. Most of these functionalities are even used extensively in the Linux kernel code. So why can't we use them, too? 

By: Nileshe Govande

The author is a Linux enthusiast and can be contacted at nileshgovande@yahoo.com. His areas of interest include Linux systems software, application development and virtualisation. He is currently working with the LSI Research & Development Centre, Pune.

THE COMPLETE MAGAZINE ON OPEN SOURCE



Your favourite Linux Magazine is
now on the Web, too.

LinuxForU.com

Follow us on Twitter @LinuxForYou